Estructuras de datos Básicas

ADT para representar colecciones

- Las colecciones de objetos se requieren frecuentemente en el desarrollo de software
- Operaciones frecuentes con colecciones:
  - agregar/eliminar items de la colección. Poder almacenar cantidades arbitrarias de items.
  - contener items de cualquier tipo: Genéricos
  - iterar sobre los items de la colección: Iteradores

Tipos de colecciones

- Bolsa (Bag)
  Solo se agregan items
  El orden de los items no importa
- Cola (Queue / FIFO queue)
  Se agregan items y se remueven preservando el orden de llegada
- Pila (Stack / LIFO queue)
  Se agregan items y se remueven en el orden inverso de llegada

APIs: Bag

public class **Bag<Item>** implements Iterable<Item>		
	Bag()	// Crear Bag vacío
void	add(Item item)	// Agregar un item
boolean	isEmpty()	// está vacío?
int	size()	// Número de items

APIs: Queue

public class Queue**<Item>** implements Iterable<Item>		
	Queue()	// Crear Queue vacío
void	enqueue(Item item)	// Agregar un item
Item	dequeue()	// Remover item al comienzo de la cola
boolean	isEmpty()	// está vacía?
int	size()	// Número de items

APIs: Stack

public class Stack**<Item>** implements Iterable<Item>		
	Stack()	// Crear una pila vacía
void	push(Item item)	// Agregar un item
Item	pop()	// Remover el último item agregado
boolean	isEmpty()	// está vacío?
int	size()	// Número de items

Nota:
Importar el paquete
`import edu.princeton.cs.algs4.Stack;`
Las bibliotecas de Java contienen una clase ligeramente distinta en el paquete `java.util.Stack`.

Ejemplo: Pila de capacidad fija

```
public class PilaStringArreglo {

	private String[] pila;
	private int n;

	public PilaStringArreglo(int max) {
		pila = new String[max];
	}

	public void push(String s) {
		pila[n++] = s;
	}

	public String pop() {
		return pila[--n];
	}

	public boolean isEmpty() {
		return n==0;
	}

	public int size() {
		return n;
	}

}
```

Resolver:

- Que pasa si se añaden más de `max` elementos?
- Si se hace `pop()` y la lista está vacía?
- Que hacer si se necesitan datos de otro tipo?

Genéricos ó Tipos parametrizados

- Permiten indicar que las clases utilizan parámetros de un tipo variable. De esta forma, las colecciones se pueden implementar de forma “genérica”, para cualquier tipo de dato.
- El parámetro de tipo se indica entre <> al momento de invocar el constructor.
- Ventajas:
  - El compilador hace chequeo de tipo con los datos contenidos en la colección
  - Se elimina el casting al sacar datos de la colección

Pila Genérica de tamaño fijo

```java
public class PilaGenericaArreglo<T> {

	private T[] pila;
	private int n;

	@SuppressWarnings("unchecked")
	public PilaGenericaArreglo(int n) {
		// NOTA: Java no permite arreglos de tipo generico
		pila = (T[]) new Object[n];
	}

	public void push(T s) {
		pila[n++] = s;
	}

	public T pop() {
		return pila[--n];
	}

	public boolean isEmpty() {
		return n==0;
	}

	public int size() {
		return n;
	}

}
```

Genéricos: Ejemplo de uso

```
Stack<String> pila = new Stack<>();
pila.push(“Hola”);
…
String dato = pila.pop();
```

Autoboxing

- Los tipos primitivos no son tipos de referencias.
- Java define clases que contienen datos de tipo primitivo: Integer, Long, Short, Byte, Boolean, Double, Float, Character.
- El compilador automáticamente convierte tipos primitivos a tipos de referencia y viceversa.

Por qué no manejar entonces todas las variables usando las wrapper classes?

Ejemplo autoboxing

```
Queue<Integer> cola = new Queue<>();
cola.enqueue(Integer.valueOf(1));
cola.enqueue(1);
…
int a = cola.dequeue();
int b = cola.dequeue();
```

Iteradores para Colecciones

Implementación es dependiente del lenguaje.

Consideraremos los casos:

1. Java
2. Python
3. C#

1. Java:

Implementan la interfaz:

```
public interface Iterable<T>
{
    Iterator<T> iterator();
}
```

y devuelven un iterador sobre la colección:

```
public interface Iterator<T> {
    boolean hasNext();
    T next();
    void remove();
}
```

Iteración mediante la sentencia foreach

- Iterar sobre todos los items de una colección es una operación muy común.
- Para facilitar esta operación se implementa la interfaz `Iterable`.
- Cuando un ADT es Iterable se puede utilizar esta sintaxis simplificada

```
for(Item i: coleccion) {
   StdOut.println(i);
}
```

Equivalencia de la sentencia foreach

```
for(Iterator i=coleccion.iterator(); i.hasNext(); ) {
    Item x = i.next();
    // Usar el item x
}
```

Iterador reverso para la pila

```
public class PilaGenericaArreglo<T> implements Iterable<T> {

      …

      public Iterator<T> iterator() {
            return new IteradorReverso();
      }


      private class IteradorReverso implements Iterator<T> {

            private int pos=n;

            @Override
            public boolean hasNext() {
                  return pos>0;
            }

            @Override
            public T next() {
                  return pila[--pos];
            }

      }


}
```

```
public static void main(String[] args) {
      PilaGenericaArreglo<Integer> p = new PilaGenericaArreglo<>(10);
      …
      for(Integer item: p) {
            StdOut.println(item);
      }

}
```

2. Python

Se tienen dos mecanimos principales

- Iterable / Iterator
- Funciones generadoras

Python: Iterable / Iterator

- Una clase que implementa el método `__iter__` es iterable.
- El método `__iter__` debe retornar un iterator el cual debe tener el método `__next__`.

Python: Iterable / Iterator

```
class Stack:

    def __init__(self):
        self._datos = []

    def __iter__(self):
        self._pos = self.size()
        return self

    def __next__(self):
        if self._pos>0:
            self._pos = self._pos-1
            return self._datos[self._pos]
        else:
            raise StopIteration()
```

Python: Función generadora

```
class Stack:

    def __init__(self):
        self._datos = []

    def __iter__(self):
        for i in range(len(self._datos) - 1, -1, -1):
            yield self._datos[i]
```

3. C#

```
class Stack<T> : IEnumerable<T> {

    private T[] datos;
    private int n;

    public Stack(int n) {
        datos = new T[n];
        this.n = 0;
    }

    public IEnumerator<T> GetEnumerator() {
        for (int i = n - 1; i >= 0; i--)
            yield return datos [i];
    }


    // Necesaria para mantener compatibilidad

    // con el IEnumerator no genérico

    IEnumerator IEnumerable.GetEnumerator() => GetEnumerator();


}
```

Implementación de colecciones

- Varias alternativas
  - Con arreglos
  - Con tipos genéricos
  - Con listas